
The Stale Comment: A Dormant-Frame Assumption Left 19 ms of Dead Bandwidth in Every Optimizer Step

Claude Opus 5 (planning author), OPHIS¹

Abstract

This is a planning paper: its primary output is a ranked, scored, pre-registered experiment program, and the analysis exists to justify it. Two independent proposer agents — one reasoning from kernels, one from architecture — converged on the same defect without knowing of each other. The n-gram sparse RMSProp step decays its entire state tensor on every optimizer step, and the code says why: a comment reading “Step-time cost does not affect val_bpb under STOP_MODE=steps.” That comment is true of the fixed_steps_2000 frame, which this campaign’s protocol marks dormant. Under the only live frame — 300s of wall clock — it is false, and the block was therefore never optimized. At $2.097\text{M} \times 384$ fp32 per half-table across roughly ten sites, the full-table lerp_moves on the order of 32 GB per step, which at H200 bandwidth is ≈ 19 ms of a ≈ 150 ms step. It is batch-independent, and the campaign’s own throughput model has an unexplained batch-independent term of 15.0 ms. We treat that coincidence as the block’s leading explanation and make it the first experiment. We also report two corrections to the campaign’s own record. First, the registered baseline of 0.93493 is contaminated: it re-measured the adopted configuration during a session in which 94% of runs completed ≈ 1147 optimizer steps instead of ≈ 2000 , and it lands $+0.00539$ worse than the same configuration measured clean. The reference points are the naive baseline 1.0301 and the campaign SOTA 0.927183. Second, the execution host changed; pip’s de-

fault install initially resolved torch 2.13.0, under which the campaign’s own record says NGRAM_SPARSE_GRAD is a -9.3% throughput win rather than the step-loss it was on 2.9.1 — but the adopted FA3 kernel wrapper (varunneal/flash-attention-3) ships no prebuilt variant past torch 2.12, so the environment was pinned to torch 2.12.x instead, and the 2.13-specific throughput claim does not apply on this host. Six mechanisms are scored on seven axes; adversarial critique kills two outright on evidence gathered while writing this paper.

1. Next Experiments

Every proposal is a MECHANISM (new code, new causal structure). The plateau rule in docs/EXPERIMENT_WORKFLOW.md forbids knob experiments here: the previous session ran eleven knobs for zero adoptions, and its own measurements showed why — intrinsic quality effects in this architecture sit at $\sim 10^{-3}$ against a gate of $2.4\text{--}4.1 \times 10^{-3}$.

1.1. The arithmetic every proposal is sized against

Token law (validated out of sample, scoped to fixed batch): $\text{val_bpb} = a - b \ln(\text{tokens})$, $b \approx 0.06$. A fractional throughput gain f is worth $-0.06 \ln(1+f)$. Break-even for a mechanism costing s step time is $0.06 \cdot (-\ln(1-s))$. The gate is ≈ 0.0024 ; we size against 0.003 as the safe bar. Therefore $f \approx 5\%$ clears the gate on throughput alone, and any mechanism costing more than $\sim 1.5\%$ step time is dead on arrival against a ~ 0.0007 intrinsic ceiling.

1.2. M1 — Exact lazy second-moment state (rank 1)

Mechanism. train.py:2572 and 2579–2581 run lerp_ over the full state tensor every step, while at most $B \cdot T = 147,456$ of 2,097,152 rows were touched. Keep a host scalar $s_t = \beta_2^t$ and store $\tilde{v} = v/s_t$: global decay becomes advancing s_t (no kernel), touched rows accu-

¹Autoresearch reimplement campaign. Correspondence to: Claude Opus 5 (OPHIS) <baiyuzhu@mit.edu>.

multilate with $\alpha = (1 - \beta_2)/s_t$, read-back multiplies by s_t , and the table is renormalized every $K=256$ steps to bound fp32 range. Why we believe it works: this is arithmetic, not a hypothesis — the update is mathematically identical, so it cannot cost intrinsic quality; and the predicted saving (≈ 19 ms) coincides with the campaign’s measured, previously unexplained batch-independent term (15.0 ms in $t_{\text{step}} = 15.0 + 1.875B$, confirmed to within 3.6% on every uncontended arm). Predicted: $f \approx 13\text{--}15\% \Rightarrow -0.0073$ to -0.0084 , i.e. $3\times$ the gate. Falsifier (pre-registered): (i) a CUDA-event timer on the decay block must read ≥ 8 ms before launch, else abandon; (ii) after the change that timer must read ≤ 1 ms; (iii) reconstructed $v = \tilde{v}s_t$ must match the reference path to $<10^{-5}$ relative at steps 1, 255, 256, 257 (guards the renormalization boundary); (iv) paired $n=3$ improvement ≥ 0.003 . Cost: ~ 12 -line diff, 5 GPU-min screen + 55 GPU-min. Risk: void if the champion runs rowwise=1 (state is then $(\text{rows}, 1)$ and the sweep is ~ 0.05 ms) — the 5-minute screen resolves this.

1.3. M2 — Factorized value embeddings, $384 \rightarrow 96$ (rank 2)

Mechanism. Each VE table is `nn.Embedding(rows, 384)`. Replace with rank 96 plus a shared $96 \rightarrow 384$ up-projection (ALBERT-style factorization, [arXiv:1909.11942](https://arxiv.org/abs/1909.11942), applied here to hash-bucket tables). Gather and capture traffic drop $4\times$; optimizer state drops $4\times$; table VRAM drops $4\times$. The added op is one well-shaped GEMM ($M=147456, K=96, N=384$), the opposite of the small GEMMs the profile blames. Why we believe it works, grounded in our own data: the campaign measured that bigger tables regress monotonically across 5/5 arms, with an isolated intrinsic capacity benefit of only ≈ 0.0007 . Per-bucket capacity is already past its optimum in this frame, so cutting per-row rank moves along the direction our own evidence endorses while the shared projection pools statistical strength across under-trained buckets. Predicted: $f \approx 8\text{--}28\% \Rightarrow -0.0046$ to -0.0148 . Also frees $\sim 12\text{--}24$ GB, which is the specific reason CUDA graphs died earlier (~ 25 GB of graph pools atop ~ 30 GB of tables), so it unblocks a previously VRAM-refuted lever. Falsifier: step time must drop $\geq 8\%$ or abort before spending seeds; then paired $n=3 \geq 0.003$. Risk: the only proposal here that changes the model, so it carries genuine intrinsic risk; retreat path is rank 192.

1.4. M3 — Visit-count-decayed row learning rate (rank 3)

Mechanism. Give each row a visit counter and scale its update by $1/(1 + n_{\text{row}}/c)$ — in the optimizer, not the forward pass. Why we believe it works: three separately established local facts compose. The frame sits at the 2-epoch boundary; the n-gram tables are schedule-exempt (NGRAM_WARMDOWN_RATIO= 0) and take full-rate steps to the last step while every other group anneals to 0.05 of peak; and the n-gram module is the sole source of data-limitation sensitivity (-0.029 at 10 shards, $+0.086$ at 5). Second-epoch updates on repeated n-grams are therefore memorization at full rate. This is distinct from the refuted NGRAM_BACKOFF_KAPPA, which shrinks the forward value of rare rows and anneals the wrong way in time; this shrinks the update of over-visited rows and leaves the forward path untouched. Falsifier — dose-response, the strongest available here: the effect must be larger at 5 shards than at 10, recovering ≥ 0.02 of the recorded $+0.086$ harm. If the 5-shard replication shows no recovery, the second-epoch-memorization story is falsified even if the 10-shard number is positive. Cost: ~ 10 lines once M1’s per-row path exists; 55 GPU-min.

1.5. M4 — Per-row confidence column in the n-gram gate (rank 4)

Mechanism. The gate is $2\sigma(W_g x)$ — a function of the hidden state only; the retrieved row never influences its own admission. Widen each table by one column, initialized to zero (exact step-0 identity), and add it pre-sigmoid. A collided or rare row can then learn a negative bias and switch itself off, instead of the model having to suppress the row globally and destroy whatever signal it carries. Provenance: Engram ([arXiv:2601.07372](https://arxiv.org/abs/2601.07372)) uses retrieved memory as key/value in its fusion gate; Engram-Nine ([arXiv:2601.16531](https://arxiv.org/abs/2601.16531)) reports that the bottleneck in this family is gating credit assignment, not lookup precision. OPHIS has exactly the defective gate that paper diagnoses. Cost: $+0.26\%$ step time $\Rightarrow$ break-even 0.0003. Falsifier: $\text{val_bpb} \leq 0.92850$ at $n=3$; plus a mechanism-engagement diagnostic — the learned column must be non-degenerate (std > 0.2 , $> 5\%$ of rows below -1). A degenerate column means the mechanism was never used, which is an uninformative null rather than a refutation.

1.6. M5 — Grouped / launch-fused sparse optimizer (rank 5)

Mechanism. The block runs eager and per-parameter, issuing ~ 15 – 20 kernels per param across ~ 10 params. Group tables by size class into one contiguous block, run one torch.unique over the concatenated keys, and `_foreach_` the rest: ~ 150 – 200 launches $\rightarrow \sim 15$ – 25 . Falsifier: the block timer must show $\geq 3\times$ reduction, and parameter trajectories must match the current path to 10^{-3} relative (summation order may change; semantics must not). Note: M5 and M1 target the same block. Run M1 first — it is exact and 12 lines — then re-measure before committing to M5.

1.7. M6 — Drop the V projection on short-window layers (rank 6)

Mechanism. Our confirmed structural win was halving the short-attention span, on the reasoning that the n-gram memory already covers local recall — attention span and n-gram memory are substitutes. The strong form of that claim: on T layers, delete `c_v` and let $v = \text{gate} \cdot v_e$ directly, removing 3 of 12 768×768 GEMMs plus their backward. Negative step cost ($\approx -3\%$, worth $+0.0018$ before any intrinsic effect). Falsifier: $\text{val_bpb} \leq 0.92800$ at $n=3$; if it lands above 0.9310 the substitution hypothesis has a hard boundary and the span result was span-specific — itself worth knowing.

1.8. Killed by adversarial critique

Recording casualties is protocol; a slate with none means the critic was weak.

- FlashAttention-4 on Hopper. A proposer correctly found that `train.py:197–201` guards fa4 behind SM100+ while upstream FA4 supports SM90 — a real finding. Killed on logistics measured while writing this paper: `github.com` times out from this host (129s, connection timed out), so `flash-attn-4` and its CuTeDSL dependency cannot be installed. PyPI and `hf-mirror.com` are reachable; GitHub is not. Re-open only if a mirror is found.
- Lngam latent-code memory ([arXiv:2605.24869](https://arxiv.org/abs/2605.24869)). Highest novelty on the slate, but its reference implementation is on GitHub, which is unreachable; a from-paper reimplement is ~ 150 lines touching hash bookkeeping, a new optimizer group, and new sparse-grad sites, and the proposer’s own simplification (freezing the projection to random LSH) may destroy the learned-symbol property that makes the method work. Deferred,

Table 1. Scores, 1–5 per axis. nov=novelty, prov=provenance, val=validity, imp=expected impact, rel=reliability, feas=feasibility/cost, fals=falsifiability.

	nov	prov	val	imp	rel	feas	fals
M1 lazy state	4	5	4	5	5	5	5
M2 factorized VE	4	5	4	5	3	3	5
M3 visit decay	4	4	4	4	3	5	5
M4 row confidence	4	5	4	4	3	4	4
M5 grouped optim.	3	5	4	4	4	3	5
M6 drop V on T	4	3	3	3	3	5	5

not refuted.

- Host-side `cu_seqlens`. Removes a real per-step device sync, but the proposer’s own estimate (-0.0012 to -0.0029) straddles the gate and it does not fix the registered capability gap (fa3 still infers sequence count from buffer length). Demoted to a rider on whichever of M1–M2 lands.
- Side-stream optimizer overlap. Silent cross-stream races would corrupt training in a way `val_bpb` alone may not reveal, and it graph-breaks under `fullgraph=True`. Same target as M1/M5 at far higher risk. Deferred.

1.9. Execution order and combination logic

Round 0 (5 GPU-min, not an experiment): one instrumented 60-step run with `torch.cuda.Event` around the decay block, the hash chain, backward, and the optimizer step; print rowwise, `NGRAM_SPARSE_GRAD`, and `exp_avg_sq.numel()`. This decides M1 vs M5 and can kill either before 55 GPU-min is spent. It is the highest expected value per minute on the list precisely because it removes experiments.

Then $M1 \rightarrow (M3, M4 \text{ in parallel}) \rightarrow M2 \rightarrow M6$. M3 is nearly free once M1’s per-row gather path exists. M4 and M3 are the forward and backward halves of one correction — learned admission and update braking — and we predict the naive stack loses first, because both suppress the same over-visited rows and double-count. The refinement path is registered in advance: (i) exempt the confidence column from the visit brake, (ii) activate the brake only after 50% progress, (iii) re-tune c . Report the trajectory, not just the endpoint; which stage carries the effect is the scientific yield even if the endpoint is null.

2. Two Corrections to the Campaign Record

The registered baseline is contaminated. reconciliation.json records 0.93493 ($n=10$) for the adopted configuration. The same configuration, same frame, measured on an uncontended host, gives 0.929541 ($n=10$) — a gap of +0.005390. The contaminating session completed ≈ 1147 optimizer steps in 45 of 48 runs where clean runs complete ≈ 2000 ; under the token law a 1147/2000 step ratio alone predicts +0.033, so the direction and rough scale are accounted for. Gating future work against 0.93493 would anchor the campaign to a number that exists only because GPUs were over-subscribed. The reference points are the naive baseline 1.0301 and the SOTA 0.927183.

The host and toolchain changed, and an initial torch-2.13 read was corrected before use. Execution moved to a host with eight idle H200s. pip install torch initially resolved 2.13.0+cu130, under which the campaign’s own record states NGRAM_SPARSE_GRAD was a clean -9.3% step-time win (versus a step-loss on 2.9.1) — but the adopted FA3 wrapper (varunneal/flash-attention-3) publishes prebuilt variants only through torch 2.12.x/cu126/cu128/cu130, with no 2.13 build, so the environment was pinned to torch 2.12.x instead. The 2.13-specific throughput claim is therefore inapplicable on this host and must not be relied on when choosing whether NGRAM_SPARSE_GRAD is held on or off across the paired arms of any proposal above; the flag must simply be held identical in both arms of every experiment, or these become knob tests in disguise.

3. Conclusion

The defect at the centre of this plan is not an algorithmic subtlety. It is a comment that was true when written, became false when the campaign’s decision frame changed, and was never revisited — leaving a full-table bandwidth sweep in the hot path of every optimizer step. Two agents reasoning from unrelated starting points found it independently, and its predicted cost coincides with a term this campaign had already measured and could not explain. That is the strongest prior we have had on this frontier in several blocks, and it costs twelve lines to test.